Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Binary Robust Positioning Patterns with Low Redundancy and Efficient Locating Algorithms

Yeow Meng Chee*† Duc Tu Dao* Han Mao Kiah*† San Ling*† Hengjia Wei*†

Abstract

A robust positioning pattern is a large array that allows a mobile device to locate its position by reading a possibly corrupted small window around it. This paper provides constructions of binary positioning patterns, equipped with efficient locating algorithms, that are robust to a constant number of errors and have redundancy within a constant factor of optimality.

Our construction of binary robust positioning sequences has the least known redundancy amongst those explicit constructions with efficient locating algorithms. On the other hand, for binary robust positioning arrays, our construction is the first explicit construction whose redundancy is within a constant factor of optimality. The locating algorithms accompanying our constructions run in time cubic in sequence length or array dimensions.

1 Introduction

Consider the problem of determining the *global* position of a mobile device in a wide environment by simply sensing a small *local* area around the device. This problem is fundamental in robotics and have practical applications in areas, such as robot localization [15], camera localization [16], 3D surface imaging by structured light [7], projected touchscreens [5] and smart styli [1].

A classic solution is via the use of positioning patterns. A *positioning pattern* is a large array, in which all contiguous subarrays of size $n_1 \times n_2$ are distinct from each other. The dimension $n_1 \times n_2$ is called the *strength* of the positioning pattern. In the special where $n_1 = 1$, we refer to the one-dimensional positioning pattern as *positioning sequence*. In practical applications, the positioning pattern is embedded in the wide area, and the mobile device reads a small *window* of the pattern, i.e., a subword of length n or a subarray of size $n_1 \times n_2$. Then due to the uniqueness of the window's subpattern, we are able to infer the position of the device. Positioning patterns have been extensively studied [12, 10, 14, 13, 6] and classical examples include de Bruijn sequences, m-sequences, perfect maps (also known as de Bruijn tori) and pseudorandom arrays.

In reality, physical devices are prone to error and we want to locate a device even when we read a small window erroneously. To this end, we study a class of positioning patterns, called *robust positioning patterns*, where the subpatterns in distinct windows are far apart from each other in the Hamming metric. In other words, the subpatterns in all windows of a robust positioning pattern form an error-correcting code.

The study on robust positioning can be traced back to the work of Kumar and Wei [10] on the minimum distance of partial periods of an m-sequence. Recently, Berkowitz and Kopparty [3] presented explicit constructions of robust positioning patterns, along with *efficient* locating algorithms. In particular, Berkowitz and Kopparty constructed high-rate q-ary robust positioning patterns (both one- and two-dimensional patterns) that locate a position even if a constant *fraction* of entries in a window are erroneous. In the regime where a constant *number* of errors are present in a window, the authors provided constructions with redundancy within a constant factor of optimality when the alphabet size is sufficiently large. When $q = 2$, the authors provided one-dimensional positioning patterns robust to a constant number of errors, but the result relies on the existence of suitable Mersenne-like primes. Efficient positioning in binary two-dimensional patterns robust to a constant number of errors remains open.

In this paper, we focus on *binary* positioning patterns that are robust to a constant *number* of errors and have accompanying efficient locating algorithms. Without relying on any unproven conjectures, we provide constructions for both one- and two-dimensional patterns whose redundancies are within a constant factor of optimality (in fact, we reduce the constant factor in the case for the one-dimensional pattern). Along with these patterns, we propose efficient locating algorithms with complexity $O(n^3)$ or $O((n_1 n_2)^3)$, where n or $n_1 \times n_2$ is the strength of the pattern. Our construction is based

*School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore. Emails: {ymchee, daoductu001, hmkiah, lingsan, hjwei}@ntu.edu.sg

†The research of Y. M. Chee, H. M. Kiah, S. Ling, and H. Wei was supported in part by the Singapore Ministry of Education under grant MOE2015-T2-2-086.

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

on d-auto-cyclic vectors, Reed-Solomon codes and Gray codes. In the full version of this paper, we determine the maximum length of robust positioning sequences for all alphabet sizes when the distance is large enough.

2 Preliminaries and Contributions

For integers i, j with $i < j$, let $[i, j]$ denote the set of integers $\{i, i+1, i+2, \ldots, j\}$. For an integer $N \geq 2$, let $[\![N]\!]$ denote the set $[0, N-1]$. Let Σ be an alphabet with q symbols and we index an array of dimension $N_1 \times N_2$ using the set $[\![N_1]\!] \times [\![N_2]\!]$. In particular, for an array $\mathbb{A} = (a_{ij}) \in \Sigma^{N_1 \times N_2}$, we use $\mathbb{A}[i, i+n_1-1][j, j+n_2-1]$ to denote the $n_1 \times n_2$ cyclical contiguous subarray of $\mathbb{A}$ whose top-left cell is a_{ij}; in the one-dimensional case, for a sequence $\boldsymbol{s} = s_0 s_1 s_2 \cdots s_{N-1} \in \Sigma^N$, we use $\boldsymbol{s}[i, i+n-1]$ to denote the length-n cyclical contiguous subword of $\boldsymbol{s}$ starting at s_i.

Denote the Hamming weight of a matrix $\mathbb{V}$ by $w_H(\mathbb{V})$. For two matrices $\mathbb{V}$ and $\mathbb{W}$ of the same dimension $N_1 \times N_2$, let $\text{agree}(\mathbb{V}, \mathbb{W})$ be the number of positions at which the corresponding entries are the same and $d_H(\mathbb{V}, \mathbb{W})$ be the Hamming distance between them. In other words, $\text{agree}(\mathbb{V}, \mathbb{W}) + d_H(\mathbb{V}, \mathbb{W}) = N_1 N_2$.

A q-ary *robust positioning array* (RPA) of strength $n_1 \times n_2$ and distance d is an array $\mathbb{A}$ over Σ in which every pair of rectangular subarrays of size $n_1 \times n_2$ is of Hamming distance at least d apart. In other words, $d_H(\mathbb{A}[i, i+n_1-1][j, j+n_2-1], \mathbb{A}[i', i'+n_1-1][j', j'+n_2-1]) \geq d$ for all distinct $(i, j), (i', j') \in [\![N_1 - n_1 + 1]\!] \times [\![N_2 - n_2 + 1]\!]$. We denote such array as an $(n_1 \times n_2, d)_q$-RPA. For an $(n_1 \times n_2, d)_q$-RPA of size $N_1 \times N_2$, define its *rate* to be $(\log_q N_1 N_2)/(n_1 n_2)$ and define its *redundancy* to be $n_1 n_2 - \log_q(N_1 N_2)$. Given n_1, n_2 and d, we are interested in the minimum redundancy of an $(n_1 \times n_2, d)$-RPA of size $N_1 \times N_2$ and denote this quantity by $\text{red}(n_1 \times n_2, d)$. When $q = 2$, we suppress q in the notation.

Since all the subarrays of dimension $n_1 \times n_2$ in an $(n_1 \times n_2, d)$-RPA form a binary error-correcting code of size $(N_1 - n_1 + 1)(N_2 - n_2 + 1)$ with minimum distance d, we have the following bound on $\text{red}(n_1 \times n_2, d)$.

PROPOSITION 2.1. (SPHERE-PACKING BOUND) *For all n_1, n_2 and d, we have that*

$$\text{red}(n_1 \times n_2, d) \geq t \log(n_1 n_2) + O(1),$$

where $t = \lfloor (d-1)/2 \rfloor$.

In the special where $N_1 = n_1 = 1$, we refer to the one-dimensional q-ary robust positioning array of strength $1 \times n$ and distance d as *robust positioning sequence* (RPS) and denote it as $(n, d)_q$-RPS. The maximum length of an $(n, d)_q$-RPS is denoted by $P_q(n, d)$. So the minimum redundancy $\text{red}_q(n, d) = n - \log_q P_q(n, d) \geq t \log n + O(1)$, where $t = \lfloor (d-1)/2 \rfloor$.

In this work, for fixed values of d, we are interested in constructing (A) (n, d)-RPS with redundancy at most $C_s d \log n + o(1)$ for some constant C_s, and (B) $(n_1 n_2, d)$-RPS with redundancy at most $C_a d \log(n_1 n_2) + o(1)$ for some constant C_a. Additionally, we require our constructions to have efficient locating algorithms.

2.1 Previous Work

One-dimensional RPS. De Bruijn sequences and m-sequences are examples of positioning sequences. Decodable de Bruijn sequences can be found in Mitchell *et al.* [13]. In 1992, Kumar and Wei [10] studied m-sequences with error-correcting ability. Using random irreducible linear feedback shift register sequences, they showed the existence of a sequence of length $2^n - 1$ in which any pair of subwords of length approximately $n + d \log n$ has Hamming distance at least d (and at most $2d$) for $d \leq \sqrt{n}$. In 2008, Hagita et al. [9] presented constructions for almost optimal $(n, 2)$-RPS and the $(n, 3)$-RPS. However their constructions are based on a conjecture on the existence of a certain type of primitive polynomials. In these constructions, no efficient locating algorithm was provided.

Recently, Berkowitz and Kopparty [3] presented explicit constructions of robust positioning sequences with efficient locating algorithms. For $q = n + 1$, they constructed positioning sequences of length $q^{n-3d-O(1)}$. For binary robust positioning sequences, they proposed an "augmented" code concatenation scheme and constructed a class of binary sequences with constant relative distance δ. They also studied binary sequences with constant distance d. However, their result relies on an open conjecture on the existence of suitable Mersenne-like primes. Furthermore, assuming the correctness of the conjecture, the redundancy of the (n, d)-RPS in their construction is at least $9d \log n$.

More recently, Wang *et al.* [17] proved that the Gilbert-Varshamov (GV) bound can be attained. Specifically, using the probabilistic method, they showed that $\text{red}(n, d) \leq d \log n + \log n + o(1)$. In addition, Wang *et al.* studied the problem under a probabilistic noise model and provided efficient algorithms to locate the position with high probability.

Two-dimensional RPA. When $d = 1$, perfect maps and pseudorandom arrays have been studied extensively as the two-dimensional generalization of de Bruijn sequences and m-sequence [6, 14, 12]. For large values of d, Bruckstein *et al.* [4] constructed a class of binary RPAs that correctly finds the location provided

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

less than a quarter of the bits in each row and less than half of the bits in each column are in error. Berkowitz and Kopparty [3] provided efficient constructions of high rate, constant relative distance RPAs over large q-ary alphabets. In the same paper, they mentioned that these q-ary arrays can be used to construct binary RPAs of high rate and constant relative distance. They also remarked that their methods were unable to construct RPAs with optimal redundancy when the distance is constant.

2.2 Our Contributions. We provide explicit constructions for RPSs and RPAs with efficient locating algorithms. The locating algorithms run in time cubic in sequence length or array dimensions, independent of the distance d. Our contributions are as follow.

(A) In Section 3, we provide an explicit construction of the (n, d)-RPS with redundancy at most $3d \log n + 6.5 \log n + O(1)$, along with an efficient locating algorithm of complexity $O(n^3)$, for fixed d. This improves on Berkowitz and Kopparty's [3] construction that requires $9d \log n$ redundancy.

(B) Let $n_1 = O(n_2^t)$ with t and d fixed. In Section 4, we provide an explicit construction of the (n_1, n_2)-RPA with redundancy at most $4.21d \log(n_1 n_2) + 36.89 \log(n_1 n_2) + o(1)$ and an efficient locating algorithm of complexity $O((n_1 n_2)^3)$. This is the first infinite family of RPAs whose redundancy is within a constant factor of optimality that have efficient locating algorithms.

2.3 Our Approach. We describe the high-level ideas behind our constructions of robust positioning arrays. Following Berkowitz and Kopparty [11], we pick a q-ary code $\mathcal{C}$ whose blocklength corresponds to the window length and we concatenate the codewords of $\mathcal{C}$ in some ordering to obtain our RPS. Hence, whenever the window coincides with a possibly erroneous codeword, we simply leverage on the error-correcting capability of $\mathcal{C}$ to locate the window. The main challenge comes when the window does *not* coincide with a codeword. To overcome this, we borrow the following tools.

(i) *Gray codes.* We use Gray codes to order the codewords of $\mathcal{C}$ so that certain windows of the sequence are of high Hamming distance apart. In fact, this method was used by Berkowitz and Kopparty [11] to construct q-ary RPS of high rates.

(ii) *Markers.* To construct binary RPS, Berkowitz and Kopparty [11] mapped the q-ary symbols of $\mathcal{C}$ to binary strings. Then they inserted short binary strings called markers into the binary sequence. These markers then allows one to locate the window's position relative to the codewords in $\mathcal{C}$. To further reduce redundancy, our construction utilizes a *d-auto-cyclic vector* as the marker.

We remark that d-auto-cyclic vectors were introduced by Levy and Yaakobi [15] in the context of DNA-based data storage. In the latter application, one objective is to design a set of primer sequences whose prefixes and suffixes satisfy certain distance property (see Yazdi et al. [19] for more details). Not surprisingly, d-auto-cyclic vectors, which are useful in the primer sequence design, turn out to be a crucial ingredient of our construction.

3 Binary Robust Positioning Sequences with Constant Distance

In this section, for fixed values of d, we propose an explicit construction for an (n, d)-RPS whose redundancy is $3d \log n + 6.5 \log n + O(1)$. As our construction is rather intricate, we first present the general ingredients required for constructing an RPS, and later provide the specific parameters to achieve the desired redundancy.

First, we review Berkowitz and Kopparty's construction [3]. Let $\boldsymbol{v}^i$ denote the concatenation of i copies of the vector $\boldsymbol{v}$, and $\boldsymbol{vw}$ denote the concatenation of two vectors $\boldsymbol{v}$ and $\boldsymbol{w}$. Let $\mathcal{C}$ be an error-correcting code of length n and minimum distance d. Berkowitz and Kopparty picked *certain* words $\boldsymbol{s}_0, \boldsymbol{s}_1, \ldots, \boldsymbol{s}_{M-1}$ from $\mathcal{C}$ and concatenated them in some order to form a long sequence $\boldsymbol{S} = \boldsymbol{s}_0 \boldsymbol{s}_1 \cdots \boldsymbol{s}_{M-1}$ of length $N = Mn$. Notice that from the choice of $\mathcal{C}$, we have that $d_H(\boldsymbol{s}_i, \boldsymbol{s}_j) \geq d$ for $0 \leq i < j \leq M-1$. In fact, via a careful choice of subwords and ordering, Berkowitz and Kopparty [3] are able to guarantee a certain distance property for *all* pairs of subwords in $\boldsymbol{S}$. We modify this technique to obtain a sequence with weaker property, where we guarantee the distance property for *some* pairs of subwords in $\boldsymbol{S}$. Formally, we have the following definitions.

DEFINITION 3.1. *Let $\boldsymbol{S}$ be a sequence. For two subwords $\boldsymbol{S}[i, i+n-1]$ and $\boldsymbol{S}[j, j+n-1]$ of length n in $\boldsymbol{s}$, we say that they start at the same modular position if $i \equiv j \pmod n$; otherwise, they start in different modular positions.*

A sequence $\boldsymbol{S}$ is called a q-ary modular robust positioning sequence of strength n and distance d, or $(n, d)_q$-MRPS for short, if

$$d_H(\boldsymbol{u}, \boldsymbol{v}) \geq d \text{ for } \boldsymbol{u}, \boldsymbol{v} \text{ in the same modular position.}$$

Next, to construct binary RPS, Berkowitz and Kopparty [3] used a short binary string called *marker* and a special mapping to transform symbols from a large

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

alphabet to binary strings. Their construction then required at least $9d \log n$ bits of redundancy and is reliant on an open conjecture about Mersenne primes. To reduce the redundancy, we utilise another marker sequence and introduce the notion of d-auto-cyclic vectors.

DEFINITION 3.2. (LEVI AND YAAKOBI [11]) *A vector $\boldsymbol{u} \in \Sigma^\ell$ is d-auto-cyclic vector if*

$$d_H(\boldsymbol{u}, 0^i \boldsymbol{u}[0, \ell - i - 1]) \geq d$$

for all $1 \leq i \leq d$.

Levy and Yaakobi provided the following construction of d-auto-cyclic vectors.

PROPOSITION 3.1. (LEVI AND YAAKOBI [11]) *Let $\ell = d\lceil \log d \rceil + 2d$. Set $\boldsymbol{u}$ to be the vector*

$$\begin{aligned} &\boldsymbol{u} = 1^d \boldsymbol{u}_0 \cdots \boldsymbol{u}_{\lceil \log d \rceil}, \text{ where} \\ &\boldsymbol{u}_i = ((1^{2^i} 0^{2^i})^d)[0, d-1]. \end{aligned} \tag{3.1}$$

Then $\boldsymbol{u}$ is a d-auto-cyclic vector.

We also introduce the notion of window weight.

DEFINITION 3.3. *Let N, k, d be positive integers such that $d < k < N$. We say a vector $\boldsymbol{v} \in \mathbb{F}_2^N$ satisfies the (d, k)-window weight limited (WWL) constraint, and is called a (d, k)-WWL vector, if $w_H(\boldsymbol{v}[i, i + k - 1]) \geq d$ for any $0 \leq i \leq N - k$.*

We are ready to present our construction.

Construction 1. Given n and d, choose k such that $\ell < k$ and $k + \ell < n$, where $\ell = d\lceil \log d \rceil + 2d$. Let $\boldsymbol{u}$ be the d-auto-cyclic vector of length ℓ and set $\boldsymbol{p} = 0^k \boldsymbol{u}$ to be a vector of length $\ell_p = k + \ell$. In addition, set $n' = n - \ell_p$. Our construction comprises the sequence $\boldsymbol{p}$ and a list of length-n' binary vectors $\boldsymbol{s}_0, \boldsymbol{s}_1, \ldots, \boldsymbol{s}_{M-1}$ satisfying the following conditions:

(P1) $\boldsymbol{s}_i$ is a (d, k)-WWL vector for $i \in [\![M]\!]$;

(P2) $\boldsymbol{s}_{i+1}[0, j-1]\boldsymbol{s}_i[j, n'-1]$ is a (d, k)-WWL vector for $i \in [\![M]\!]$ and $j \in [\![n'-1]\!]$; and

(P3) the concatenation $\boldsymbol{s}_0 \boldsymbol{s}_1 \boldsymbol{s}_2 \cdots \boldsymbol{s}_{M-1}$ is an (n', d)-MRPS.

Set $\boldsymbol{S} \triangleq \boldsymbol{p}\boldsymbol{s}_0 \boldsymbol{p}\boldsymbol{s}_1 \boldsymbol{p}\boldsymbol{s}_2 \cdots \boldsymbol{p}\boldsymbol{s}_{M-1}$.

In the next subsection, we specify the values of k and ℓ and provide an explicit method to construct $\boldsymbol{s}_i$'s. Consequently, we obtain the sequence $\boldsymbol{S}$ and show that it has the desired redundancy. Prior to this, we prove that $\boldsymbol{S}$ is indeed an (n, d)-RPS. Note that (P3) implies $\boldsymbol{S}$ is an (n, d)-MRPS. Hence, it remains to show that every two subwords in different modular positions have distance at least d. To do so, we have the following technical lemma.

LEMMA 3.1. *Consider the subword $\boldsymbol{w} = \boldsymbol{S}[i_0, i_0 + n - 1]$ in $\boldsymbol{S}$. Pick $i \in [\![n]\!]$. Then the following hold.*

(i) If $i + i_0 \equiv 0 \pmod n$, then $\boldsymbol{w}[i, i + \ell_p - 1] = \boldsymbol{p}$.

(ii) If $i + i_0 \not\equiv 0 \pmod n$, then $d_H(\boldsymbol{w}[i, i + \ell_p - 1], \boldsymbol{p}) \geq d$.

Proof. Let $\hat{i}$ be the unique integer of $[\![n]\!]$ such that $\hat{i} + i_0 \equiv 0 \pmod n$. We consider the vector $\boldsymbol{v}$, which is obtained by shifting $\boldsymbol{w}$ to the left cyclically $\hat{i}$ times. Then it suffices to show that $\boldsymbol{v}[0, \ell_p - 1] = \boldsymbol{p}$ and $d_H(\boldsymbol{v}[i, i + \ell_p - 1], \boldsymbol{p}) \geq d$ for $i \in [1, n-1]$. Suppose that $i_0 = an + \bar{i}$ where $\bar{i} \in [n]$. From Construction 1 (see Fig. 1), we have that

$$\boldsymbol{v} = \begin{cases} \boldsymbol{p}\boldsymbol{s}_a, & \text{if } \bar{i} \leq \ell_p; \\ \boldsymbol{p}\boldsymbol{s}_{a+1}[0, \bar{i} - \ell_p - 1]\boldsymbol{s}_a[\bar{i} - \ell_p, n' - 1], & \text{if } \bar{i} > \ell_p. \end{cases}$$

Hence $\boldsymbol{v}[0, \ell_p - 1] = \boldsymbol{p}$.

Now we consider $\boldsymbol{v}[i, i + \ell_p - 1]$ with $i \neq 0$. Since $\boldsymbol{s}_a$ and $\boldsymbol{s}_{a+1}$ satisfy the conditions (P1) and (P2), we can always assume that $\boldsymbol{v} = \boldsymbol{p}\boldsymbol{x}$ for some (k, d)-WWL vector $\boldsymbol{x}$ of length n'. We proceed by cases.

Case 1: $i \in [1, d]$. Then

$$\begin{aligned} \boldsymbol{v}[i + k - i, i + k - i + \ell - 1] &= \boldsymbol{v}[k, k + \ell - 1] = \boldsymbol{u}, \\ \boldsymbol{p}[k - i, k - i + \ell - 1] &= 0^i \boldsymbol{u}[0, \ell - i - 1]. \end{aligned}$$

Since $\boldsymbol{u}$ is d-auto-cyclic, we have

$$\begin{aligned} &d_H(\boldsymbol{v}[i, i + \ell_p - 1], \boldsymbol{p}) \\ &\quad \geq d_H(\boldsymbol{v}[k, k + \ell - 1], \boldsymbol{p}[k - i, k - i + \ell - 1]) \\ &\quad = d_H(\boldsymbol{u}, 0^i \boldsymbol{u}[0, \ell - i - 1]) \geq d. \end{aligned}$$

Case 2: $i \in [d + 1, \ell_p - d]$. Notice that $\boldsymbol{v} = 0^k \boldsymbol{u}\boldsymbol{x}$. Since $\ell < k$, the subword $\boldsymbol{v}[i, i + k - 1]$ should contain either the length-d prefix of $\boldsymbol{u}$ or the length-d suffix of $\boldsymbol{u}$, both of which are 1^d. So the weight of $\boldsymbol{v}[i, i + k - 1]$ is at least d. It follows that

$$\begin{aligned} d_H(\boldsymbol{v}[i, i + \ell_p - 1], \boldsymbol{p}) &\geq d_H(\boldsymbol{v}[i, i + k - 1], \boldsymbol{p}[0, k - 1]) \\ &= d_H(\boldsymbol{v}[i, i + k - 1], 0^k) \geq d. \end{aligned}$$

Case 3: $i \in [\ell_p - d + 1, n - k]$. The subword $\boldsymbol{v}[i, i + k - 1]$ is contained in $1^d \boldsymbol{x}$. Since $\boldsymbol{x}$ is a (d, k)-WWL vector, the weight of $\boldsymbol{v}[i, i + k - 1]$ is at least d. Again, we have

$$\begin{aligned} d_H(\boldsymbol{v}[i, i + \ell_p - 1], \boldsymbol{p}) &\geq d_H(\boldsymbol{v}[i, i + k - 1], \boldsymbol{p}[0, k - 1]) \\ &= d_H(\boldsymbol{v}[i, i + k - 1], 0^k) \geq d. \end{aligned}$$

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

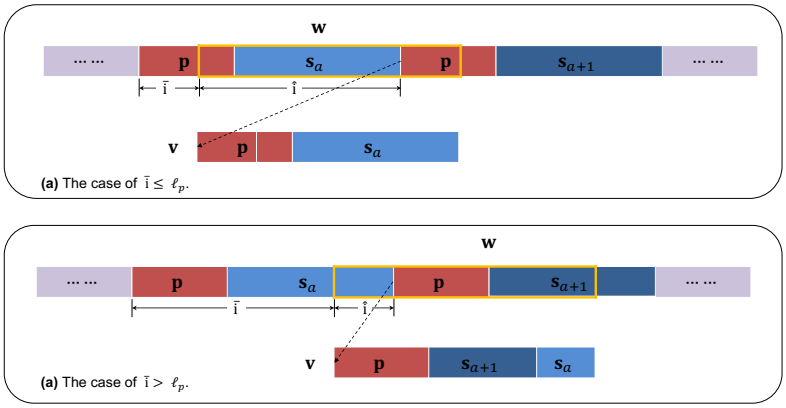


Figure 1: The vector $\boldsymbol{v}$ obtained by shifting $\boldsymbol{w}$ to the left cyclically $\hat{\imath}$ times.

Case 4: $i \in [n-k+1, n-d]$. Since $i+k-n \ge 1$ and $i+k+d-1-n \le k-1$, we have $\boldsymbol{v}[i+k, i+k+d-1] = \boldsymbol{v}[i+k-n, i+k+d-1-n] = 0^d$. Note that $\boldsymbol{p}[k, k+d-1] = \boldsymbol{u}[0, d-1] = 1^d$. It follows that $d_H(\boldsymbol{v}[i, i+\ell_p-1], \boldsymbol{p}) \ge d$.

Case 5: $i \in [n-d+1, n-1]$. Let $\delta = n-i$, then $\delta \in [1, d-1]$. We have

$$\begin{aligned}
&\boldsymbol{v}[i+k, i+k+\ell-1] \\
&\quad = \boldsymbol{v}[n+k-\delta, n+k+\ell-\delta-1] \\
&\quad = \boldsymbol{v}[k-\delta, k+\ell-\delta-1] \\
&\quad = 0^\delta \boldsymbol{u}[0, \ell-\delta-1].
\end{aligned}$$

Since $\boldsymbol{p}[k, k+\ell-1] = \boldsymbol{u}$ and $\delta \in [1, d-1]$, we have

$$\begin{aligned}
&d_H(\boldsymbol{v}[i, i+\ell_p-1], \boldsymbol{p}) \\
&\quad \ge d_H(\boldsymbol{v}[i+k, i+k+\ell-1], \boldsymbol{p}[k, k+\ell-1]) \ge d,
\end{aligned}$$

which completes the proof. □

Next, we prove that the construction is correct.

THEOREM 3.1. *Let $\boldsymbol{S}$ be the sequence constructed in Construction 1. Then $\boldsymbol{S}$ is an (n,d)-RPS.*

Proof. Let $\boldsymbol{w}_1$ and $\boldsymbol{w}_2$ be two distinct subwords of length n in $\boldsymbol{S}$. Assume that $\boldsymbol{w}_1 = \boldsymbol{S}[i, i+n-1]$ and $\boldsymbol{w}_2 = \boldsymbol{S}[j, j+n-1]$, where $i \ne j$. Since $\boldsymbol{s}_0 \boldsymbol{s}_1 \cdots \boldsymbol{s}_{M-1}$ is an (n', d)-MRPS, we have that $\boldsymbol{S}$ is an (n,d)-MRPS. Hence, $d_H(\boldsymbol{w}_1, \boldsymbol{w}_2) \ge d$ whenever $i \equiv j \pmod n$.

It remains to consider the case where $i \not\equiv j \pmod n$. Let $\hat{\imath}$ be the integer of $[n]$ such that $i + \hat{\imath} \equiv 0 \pmod n$. Hence, we have $j + \hat{\imath} \not\equiv 0 \pmod n$. Lemma 3.1 implies that $\boldsymbol{w}_1[\hat{\imath}, \hat{\imath}+\ell_p-1] = \boldsymbol{p}$ and $d_H(\boldsymbol{w}_2[\hat{\imath}, \hat{\imath}+\ell_p-1], \boldsymbol{p}) \ge d$. Hence, $d_H(\boldsymbol{w}_1, \boldsymbol{w}_2) \ge d_H(\boldsymbol{w}_1[\hat{\imath}, \hat{\imath}+\ell_p-1], \boldsymbol{w}_2[\hat{\imath}, \hat{\imath}+\ell_p-1]) \ge d$. □

3.1 Sequence Construction. Given n and d, we provide the choice of k and ℓ and construct the vectors $\boldsymbol{u}, \boldsymbol{s}_0, \boldsymbol{s}_1, \ldots, \boldsymbol{s}_{M-1}$ to satisfy conditions (P1), (P2), and (P3).

First, we set ℓ and $\boldsymbol{u}$ as in (3.1). Next, set $m \triangleq (3/2)\log n$, $k = 3m$ and $q \triangleq 2^m$. Let $\alpha_0, \alpha_1, \ldots, \alpha_{q-1}$ be the q distinct elements in $\mathbb{F}_q$ and let ϕ be an arbitrary bijection from $\mathbb{F}_q$ to $\mathbb{F}_2^m$. Set $X \triangleq \{\alpha_j \in \mathbb{F}_q : w_H(\phi(\alpha_j)) \ge d\}$. Let $r = \sum_{i=d}^{m} \binom{m}{i}$, which is the cardinality of X. Recall that $n' = n - (k+\ell)$ and our objective is to construct an (n', d)-MRPS. To this end, we require the concepts of Gray codes and Reed-Solomon codes.

DEFINITION 3.4. (GRAY CODES) *Let Σ be an alphabet with q symbols and $\mathcal{G} = (\boldsymbol{\sigma}_0, \boldsymbol{\sigma}_1, \ldots, \boldsymbol{\sigma}_{q^n-1})$ be a sequence of all the vectors in Σ^n. Then $\mathcal{G}$ is called an* (n,q)-Gray code *if any two adjacent vectors $\boldsymbol{\sigma}_i$ and $\boldsymbol{\sigma}_{i+1}$ in $\mathcal{G}$ differ in only one position.*

THEOREM 3.2. (DECODING FOR GRAY CODES [8]) *Let q and n be two positive integers. There exists an (n,q)-Gray code $\mathcal{G} = (\boldsymbol{\sigma}_0, \boldsymbol{\sigma}_1, \ldots, \boldsymbol{\sigma}_{q^n-1})$ and a decoding function $\mathrm{dec}_{\mathrm{Gray}} : \mathbb{F}_q^n \to [\![q^n]\!]$ such that $\mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{\sigma}_i) = i$ for all $i \in [\![q^n]\!]$. Furthermore, $\mathrm{dec}_{\mathrm{Gray}}$ can be computed in $O(n \log^2 q)$ time.*

THEOREM 3.3. (REED-SOLOMON CODE [18]) *Let q be a prime-power. Suppose that $k_R < n_R \le q$. Then there exists a linear code $\mathcal{C}_{\mathrm{RS}}$ of length n_R, dimension k_R and minimum distance $d_R \triangleq n_R - k_R + 1$. Furthermore, there exist encoding function $\mathrm{enc}_{\mathrm{RS}}^{(n_R,k_R)} : \mathbb{F}_q^{k_R} \to \mathcal{C}_{\mathrm{RS}}$ and decoding function $\mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)} : \mathbb{F}_q^{n_R} \to \mathcal{C}_{\mathrm{RS}}$ such that the following hold.*

(i) For all $\boldsymbol{\sigma} \in \mathbb{F}_q^{k_R}$, the k_R-prefix of $\mathrm{enc}_{\mathrm{RS}}^{(n_R,k_R)}(\boldsymbol{\sigma})$ is $\boldsymbol{\sigma}$. In other words, $\mathrm{enc}_{\mathrm{RS}}^{(n_R,k_R)}(\boldsymbol{\sigma})[0, k_R-1] = \boldsymbol{\sigma}$.

(ii) Choose $\boldsymbol{c} \in \mathcal{C}_{\mathrm{RS}}$ and suppose that $d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}) \le (d_R-1)/2$. Then $\mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)}(\bar{\boldsymbol{c}}) = \boldsymbol{c}$. Furthermore, $\mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)}$ can be computed in $O(n^3)$ time.

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In their construction for q-ary RPS, Berkowitz and Kopparty [3] used a Gray code to give an ordering to a subset of codewords in a Reed-Solomon code, and concatenated these codewords in this ordering to form the desired sequence. In our construction, we adapt the technique to obtain a family of q-ary vectors $\boldsymbol{c}_i$ such that $\boldsymbol{c}_0\boldsymbol{c}_1\cdots\boldsymbol{c}_{M-1}$ is a q-ary MRPS. Then we apply the mapping ϕ to each $\boldsymbol{c}_i$ and append short sequences 1^d in proper positions to obtain the WWL vector $\boldsymbol{s}_i$.

Specifically, set $n_R \triangleq (n' - (2d+2)d)/m$ and $k_R \triangleq n_R - 2d - 2$. Consider the Reed-Solomon code $\mathcal{C}_{\mathrm{RS}}$ from Theorem 3.3. Now we provide our construction of $\boldsymbol{s}_i$ for $i \in [\![r^{k_R}]\!]$, and consequently, the sequence $\boldsymbol{S}$.

Construction 1A. Let $M = r^{k_R}$ and $\mathcal{G} = (\boldsymbol{\sigma}_0, \boldsymbol{\sigma}_1, \ldots, \boldsymbol{\sigma}_{M-1})$ be a (k_R, r)-Gray code over X. For $i \in [\![M]\!]$, set $\boldsymbol{c}_i = \mathrm{enc}_{\mathrm{RS}}^{(n_R,k_R)}(\boldsymbol{\sigma}_i)$. Then for each $\boldsymbol{c}_i$, construct a binary vector $\boldsymbol{s}_i$ as

$$\boldsymbol{s}_i = \phi(\boldsymbol{c}_i[0])\phi(\boldsymbol{c}_i[1])\cdots\phi(\boldsymbol{c}_i[k_R-1])1^d\phi(\boldsymbol{c}_i[k_R])$$
$$1^d\phi(\boldsymbol{c}_i[k_R+1])\cdots 1^d\phi(\boldsymbol{c}_i[n_R-1]).$$

Finally, let $\boldsymbol{p} = 0^k\boldsymbol{u}$, where $\boldsymbol{u}$ is the d-auto-cyclic vector in (3.1). Construct the sequence $\boldsymbol{S}$ as

$$\boldsymbol{S} = \boldsymbol{p}\boldsymbol{s}_0\boldsymbol{p}\boldsymbol{s}_1\cdots\boldsymbol{p}\boldsymbol{s}_{M-1}.$$

We summarise in Table 1 all the parameters and notations involved in our construction (see also Fig. 2). To simplify our exposition and analysis, we assume that all parameters are integers.

Observe that each $\boldsymbol{c}_i$ is a codeword whose length-k_R prefix is $\boldsymbol{\sigma}_i$. Hence, when $j < k_R$, the symbol $\boldsymbol{c}_i[j]$ belongs to X and $w_H(\phi(\boldsymbol{c}_i[j])) \geq d$. However, when $j \geq k_R$, the symbol $\boldsymbol{c}_i[j]$ may not belong to X and the weight of $\phi(\boldsymbol{c}_i[j])$ may be less than d. So, we prepend a sequence 1^d at the head of $\phi(\boldsymbol{c}_i[j])$ for each $j \geq k_R$. Since $k = 3m \geq m + 2d$, it is easy to check that the vectors $\boldsymbol{s}_0, \boldsymbol{s}_1, \ldots, \boldsymbol{s}_{M-1}$ satisfy the conditions (P1) and (P2). For (P3), we have the following result on $\boldsymbol{S}$.

LEMMA 3.2. *The concatenation $\boldsymbol{s}_0\boldsymbol{s}_1\cdots\boldsymbol{s}_{M-1}$ is an (n', d)-MRPS.*

Proof. Since each $\boldsymbol{s}_i$ is obtained by inserting the sequences 1^d at fixed positions in the concatenation of the binary strings $\phi[\boldsymbol{c}_i[j]]$, it suffices to show that the concatenation $\boldsymbol{c}_0\boldsymbol{c}_1\cdots\boldsymbol{c}_{M-1}$ is an $(n_R, d+1)_q$-MRPS.

Assume that $\boldsymbol{w}_1$ and $\boldsymbol{w}_2$ start at position i and position j respectively. Since $\boldsymbol{w}_1$ and $\boldsymbol{w}_2$ are in the same modular position, we may assume that $\bar{m} \equiv i \equiv j \pmod{n_R}$, where $\bar{m} \in [\![n_R]\!]$. Further let $i = an_R + \bar{m}$ and $j = bn_R + \bar{m}$. We proceed by cases.

Case 1: $\bar{m} = 0$. Then $\boldsymbol{w}_1 = \boldsymbol{c}_a$ and $\boldsymbol{w}_2 = \boldsymbol{c}_b$. Since $\boldsymbol{c}_a$ and $\boldsymbol{c}_b$ belong to $\mathcal{C}_{\mathrm{RS}}$, we have that $d_H(\boldsymbol{w}_1, \boldsymbol{w}_2) = d_H(\boldsymbol{c}_a, \boldsymbol{c}_b) \geq d_R = n_R - k_R + 1 > d + 1$.

Case 2: $\bar{m} \in [1, k_R]$. So, $\boldsymbol{w}_1 = \boldsymbol{c}_a[\bar{m}, n_R-1]\boldsymbol{c}_{a+1}[0, \bar{m}-1]$. Since $\mathcal{G}$ is a Gray code, $d_H(\boldsymbol{c}_a[0, \bar{m}-1], \boldsymbol{c}_{a+1}[0, \bar{m}-1]) = d_H(\boldsymbol{\sigma}_a, \boldsymbol{\sigma}_{a+1}) \leq 1$. In other words,

$$d_H(\boldsymbol{w}_1, \boldsymbol{c}_a[\bar{m}, n_R-1]\boldsymbol{c}_a[0, \bar{m}-1]) \leq 1.$$

Similarly, we have

$$d_H(\boldsymbol{w}_2, \boldsymbol{c}_b[\bar{m}, n_R-1]\boldsymbol{c}_b[0, \bar{m}-1]) \leq 1.$$

It follows that

$$d_H(\boldsymbol{w}_1, \boldsymbol{w}_2) \geq d_H(\boldsymbol{c}_a, \boldsymbol{c}_b) - 2 \geq n_R - k_R - 1 > d + 1.$$

Case 3: $\bar{m} \in [k_R+1, n_r-1]$. We partition the interval $[0, n_R-1]$ into three pieces by setting

$$I_1 = [0, n_R - \bar{m} - 1],$$
$$I_2 = [n_R - \bar{m}, n_R - \bar{m} + k_R - 1], \text{ and}$$
$$I_3 = [n_R - \bar{m} + k_R, n_R - 1].$$

For $j \in \{1, 2, 3\}$, let $\mathrm{agree}_j = \mathrm{agree}(\boldsymbol{w}_1[I_j], \boldsymbol{w}_2[I_j])$. Then

$$\begin{aligned}\mathrm{agree}_2 + \mathrm{agree}_3 &= \mathrm{agree}(\boldsymbol{c}_{a+1}[0, \bar{m}-1], \boldsymbol{c}_{b+1}[0, \bar{m}-1]) \\ &\leq k_R - 1 \\ \mathrm{agree}_1 + \mathrm{agree}_3 &\leq |I_1| + |I_3| = n_R - |I_2| = n_R - k_R.\end{aligned}$$

Since $d_H(\boldsymbol{c}_a[0, k_R-1]), \boldsymbol{c}_{a+1}[0, k_R-1])) \leq 1$ and $d_H(\boldsymbol{c}_b[0, k_R-1]), \boldsymbol{c}_{b+1}[0, k_R-1])) \leq 1$, we have

$$\begin{aligned}\mathrm{agree}_1 + \mathrm{agree}_2 =& \mathrm{agree}(\boldsymbol{c}_a[\bar{m}, n_R-1]\boldsymbol{c}_{a+1}[0, k_R-1], \\ & \boldsymbol{c}_b[\bar{m}, n_R-1]\boldsymbol{c}_{b+1}[0, k_R-1]) \\ \leq& \mathrm{agree}(\boldsymbol{c}_a[\bar{m}, n_R-1]\boldsymbol{c}_a[0, k_R-1], \\ & \boldsymbol{c}_b[\bar{m}, n_R-1]\boldsymbol{c}_b[0, k_R-1]) + 2 \\ \leq& \mathrm{agree}((\boldsymbol{c}_a, \boldsymbol{c}_b) + 2 \leq k_R + 1.\end{aligned}$$

Summing these three inequalities yields

$$\mathrm{agree}_1 + \mathrm{agree}_2 + \mathrm{agree}_3 \leq \frac{n_R + k_R}{2}.$$

Therefore, we have

$$\begin{aligned}d_H(\boldsymbol{w}_1, \boldsymbol{w}_2) &= n_R - \mathrm{agree}(\boldsymbol{w}_1, \boldsymbol{w}_2) \\ &\geq \frac{n_R - k_R}{2} = d + 1,\end{aligned}$$

which completes the proof. □

Therefore, Construction 1A yields an (n, d)-RPS as desired. We analyse the required redundancy in the following corollary.

COROLLARY 3.1. *For sufficient large n, there is an (n, d)-RPS with redundancy at most $3d\log n + 6.5\log n + O(1)$.*

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Notation	Remark	Notation	Remark
n	the strength of the RPS	$\mathbb{F}_q$	the finite field with q elements
d	the distance of the RPS	α_j	the element in $\mathbb{F}_q$
ℓ	the length of the d-auto-cyclic vector	ϕ	a one-to-one map from $\mathbb{F}_q$ to $\mathbb{F}_2^m$
m	$m \triangleq \frac{3}{2}\log n$	X	the set of α_j such that $w_H(\phi(\alpha_j)) \geq d$
k	$k \triangleq 3m$	$\mathcal{G}$	a (k_R, r)-Gray code
ℓ_p	$\ell_p \triangleq k + \ell$	$\boldsymbol{\sigma}_i$	the i-th vector in $\mathcal{G}$
r	$r \triangleq \sum_{i=d}^{m} \binom{m}{i}$	$\boldsymbol{c}_i$	the codeword in a q-ary Reed-Solomon code of length n_R
n'	$n' \triangleq n - \ell_p$		and dimension k_R such that $\boldsymbol{c}_i[0, k_R - 1] = \boldsymbol{\sigma}_i$
q	$q \triangleq 2^m$	$\boldsymbol{s}_i$	the concatenation of the binary vectors $\phi[\boldsymbol{c}_i[j]]$, $j \in [k_R]$,
n_R	$n_R \triangleq \frac{1}{m}(n' - (2d+2)d)$		as well as the vectors $1^d\phi[\boldsymbol{c}_i[j]]$, $k_R \leq j \leq n_R - 1$
k_R	$k_R \triangleq n_R - 2d - 2$	$\boldsymbol{u}$	the d-auto-cyclic vector of length ℓ
M	$M \triangleq r^{k_R}$	$\boldsymbol{p}$	$\boldsymbol{p} \triangleq 0^k\boldsymbol{u}$
		$\boldsymbol{S}$	$\boldsymbol{S} \triangleq \boldsymbol{p}\boldsymbol{s}_0\boldsymbol{p}\boldsymbol{s}_1 \cdots \boldsymbol{p}\boldsymbol{s}_{M-1}$

Table 1: Notation Summary for Construction 1A

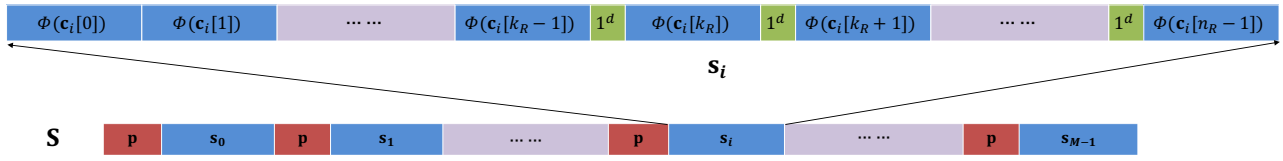


Figure 2: The sequence $\boldsymbol{S}$ in Construction 1A.

Proof. Recall that

$$\begin{aligned} r = \sum_{i=d}^{m}\binom{m}{i} &= 2^m\left[1 - \sum_{i=0}^{d-1}\binom{m}{i}\frac{1}{2^m}\right] \\ &\geq 2^m\left[1 - \exp\left(-\frac{2(m/2-d+1)^2}{m}\right)\right] \\ &\geq 2^m\left(1 - e^{-\frac{2m}{3\log e}}\right) = 2^m\left(1 - \frac{1}{n}\right). \end{aligned}$$

The first inequality comes from Hoeffding's inequality for the tail of the binomial distribution while the second one holds as $(m/2 - d + 1)^2/m > m/(3\log e)$ when m is large enough. Then we have

$$\begin{aligned} \log r &\geq m + \log\left(1 - \frac{1}{n}\right) = m + \ln\left(1 - \frac{1}{n}\right)\log e \\ &\geq m - \frac{\log e}{n-1}. \end{aligned}$$

Note that $k_R = \frac{1}{m}[n - k - \ell - (2d+2)(m+d)] = \frac{n}{m} - 2d - 5 - O\left(\frac{1}{m}\right)$. Hence the redundancy of $\boldsymbol{S}$ is

$$\begin{aligned} n - \log(nM) &= n - k_R\log r - \log n \\ &\leq n - \left(\frac{n}{m} - 2d - 5 - O\left(\frac{1}{m}\right)\right)\left(m - \frac{\log e}{n-1}\right) - \log n \\ &= 2dm + 5m - \log n + O(1) \\ &= 3d\log n + 6.5\log n + O(1). \end{aligned}$$

□

Remark. In Construction 1A, we convert the q-ary vectors into binary vectors by mapping the elements of $\mathbb{F}_q$ to the binary vectors of length m, and we append some short sequences 1^d so that the resulting sequences satisfy conditions (P1) and (P2). Alternatively, one may choose a prime power q that is at most $\sum_{i=d}^{m}\binom{m}{i}$ and map the elements in $\mathbb{F}_q$ to the binary length-m vectors with weight at least d. This approach then results in an (n, d)-RPS with redundancy $4.21d\log n + 9.53\log n + O(1)$, which is larger than that in Corollary 3.1. However, in next section, when we construct 2-D robust positioning arrays, we have to adopt this approach as it is difficult to tile some small patterns and large squares while keeping low redundancy.

3.2 Locating Algorithm. We present a locating algorithm for the subwords of the sequence $\boldsymbol{S}$ in Construction 1A. In particular, the locating algorithm corrects

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

up to $\lfloor (d-1)/2 \rfloor$ errors in $O(n^3)$ time, independent of parameter d.

Suppose that $\boldsymbol{w}$ is a subword of $\boldsymbol{S}$ that is corrupted at e positions with $e \leq (d-1)/2$. In other words, there is a unique index i such that $d_H(\boldsymbol{S}[i, i+n-1], \boldsymbol{w}) \leq (d-1)/2$ and our task is to recover i. Equivalently, if we write i as $an+\bar{i}$ with $\bar{i} \equiv i \pmod{n}$, then our task is to recover both a and $\bar{i}$. In what follows, we give a broad overview of the steps and the detailed implementation of the algorithm is provided in Algorithm 1.

(I) We determine $\bar{i}$. To do so, we determine the unique index $\hat{i}$ such that $d_H(\boldsymbol{w}[\hat{i}, \hat{i}+\ell_p-1], \boldsymbol{p}) \leq (d-1)/2$. Set $\bar{i} \in [\![n]\!]$ such that $\bar{i}+\hat{i} \equiv 0 \pmod{n}$

(II) Next, we cyclically rotate $\boldsymbol{w}$ to the left by $\hat{i}$ positions to obtain $\boldsymbol{v}$. Observe that $\boldsymbol{v}$ is the binary image obtained from either a q-ary codeword $\boldsymbol{c}_a$ or a concatenation $\boldsymbol{c}_a[n_R-j+1, n_R]\boldsymbol{c}_{a+1}[0, j]$ for some $j \in [\![n_R]\!]$. Since $\boldsymbol{v}$ is obtained via the map ϕ and prepending the string $\boldsymbol{p}$ and inserting n_R-k_R strings 1^d, we reverse this process to obtain the q-ary estimate $\bar{\boldsymbol{c}}$.

(III) Finally, depending on the value of $\bar{i}$, we apply the Reed-Solomon decoding algorithm $\mathrm{dec}_{\mathrm{RS}}$ to find either $\boldsymbol{c}_a$ or $\boldsymbol{c}_{a+1}$ or some shortened versions of these words. Therefore, we determine a and hence, obtain $i = an + \hat{i}$.

THEOREM 3.4. *Suppose $\boldsymbol{w}$ is a corrupted subword of the sequence $\boldsymbol{S}$ with exactly e errors. If $2e < d$, then Algorithm 1 can determine the position of $\boldsymbol{w}$ in the sequence $\boldsymbol{S}$ in $O(n^3)$ time.*

Proof. Suppose the corrupted subword $\boldsymbol{w}$ starts at position $\alpha n + \beta$, where $\alpha \in [\![M]\!]$ and $\beta \in [\![n]\!]$. Denote the original subword $\boldsymbol{S}[\alpha n+\beta, (\alpha+1)n+\beta-1]$ as $\boldsymbol{w}^\circ$, and so, $d_H(\boldsymbol{w}, \boldsymbol{w}^\circ) = e$. Lemma 3.1 implies that $d_H(\boldsymbol{w}[i, i+\ell_p-1], \boldsymbol{p}) \leq e$ when $i+\beta \equiv 0 \pmod{n}$, and $d_H(\boldsymbol{w}[i, i+\ell_p-1], \boldsymbol{p}) \geq d-e$ when $i+\beta \not\equiv 0 \pmod{n}$. Since $d-e > e$, the value β can be uniquely determined and we have $\bar{i} = \beta$. In order to determine α, we consider the following cases.

Case 1: $\bar{i} \in [\![\ell_p]\!]$. By shifting the original subword $\boldsymbol{w}^\circ$ to the left $\hat{i}$ times, we obtain $\boldsymbol{p}\boldsymbol{s}_\alpha$. Since shifting both $\boldsymbol{w}$ and $\boldsymbol{w}^\circ$ simultaneously does not increase the Hamming distance, we have $d_H(\boldsymbol{v}, \boldsymbol{p}\boldsymbol{s}_\alpha) = e$. After removing the sequences $\boldsymbol{p}$ and 1^d from $\boldsymbol{p}\boldsymbol{s}_\alpha$ and the corresponding subwords from $\boldsymbol{v}$, we have that $d_H(\hat{\boldsymbol{v}}, \phi(\boldsymbol{c}_\alpha[0])\phi(\boldsymbol{c}_\alpha[1]) \cdots \phi(\boldsymbol{c}_\alpha[n_R-1])) \leq e$. Next, we apply the inverse function ϕ^{-1} and observe that the Hamming distance does not increase. Therefore, $d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}_\alpha) \leq e < d/2$.

Algorithm 1 Locating algorithm for the sequence $\boldsymbol{S}$ in Construction 1A

Input: a sequence $\boldsymbol{w}$ of length n
Output: a position $i \triangleq an + \bar{i}$ such that $d_H(\boldsymbol{S}[i, i+n-1], \boldsymbol{w}) \leq (d-1)/2$

$\hat{i} \leftarrow$ unique index such that $d_H(\boldsymbol{w}[\hat{i}, \hat{i}+\ell_p-1], \boldsymbol{p}) \leq (d-1)/2$
Set $\bar{i} \in [\![n]\!]$ such that $\bar{i}+\hat{i} \equiv 0 \pmod{n}$

$\boldsymbol{v} \leftarrow$ the vector obtained by rotating $\boldsymbol{w}$ to the left cyclically $\hat{i}$ positions
$\hat{\boldsymbol{v}} \leftarrow$ the vector obtained from $\boldsymbol{v}$ by deleting $\boldsymbol{v}[0, \ell_p-1]$ and $\boldsymbol{v}[\ell_p+mk_R+(m+d)j, \ell_p+mk_R+(m+d)j+d-1]$ for all $j \in [n_R-k_R]$
$\bar{\boldsymbol{c}} \leftarrow \phi^{-1}(\hat{\boldsymbol{v}}[0, m-1])\phi^{-1}(\hat{\boldsymbol{v}}[m, 2m-1]) \cdots \phi^{-1}(\hat{\boldsymbol{v}}[m(n_R-1), mn_R-1])$

if $\bar{i} \in [0, \ell_p+mk_R-1]$ **then**
  $\boldsymbol{c} \leftarrow \mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)}(\bar{\boldsymbol{c}})$
  $a \leftarrow \mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{c}[0, k_R-1])$
else if $\bar{i} \in [\ell_p+mk_R, \ell_p+mk_R+(d+1)(m+d)-1]$ **then**
  $\bar{\boldsymbol{c}}^s \leftarrow$ the shortened codeword $\bar{\boldsymbol{c}}[0, k_R-1]\bar{\boldsymbol{c}}[k_R+(d+1), n_R-1]$
  $\boldsymbol{c}^s \leftarrow \mathrm{dec}_{\mathrm{RS}}^{(n_R-(d+1),k_R)}(\bar{\boldsymbol{c}}^s)$
  $a \leftarrow \mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{c}^s[0, k_R-1])$
else
  $\bar{\boldsymbol{c}}^s \leftarrow$ the shortened codeword $\bar{\boldsymbol{c}}[0, k_R+(d+1)-1]$
  $\boldsymbol{c}^s \leftarrow \mathrm{dec}_{\mathrm{RS}}^{(n_R-(d+1),k_R)}(\bar{\boldsymbol{c}}^s)$
  $a+1 \leftarrow \mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{c}^s[0, k_R-1])$
return $an+\bar{i}$

Since $d_R = 2d+2$, we have that $d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}_\alpha) \leq (d_R-1)/2$, applying the decoding algorithm $\mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)}$ on $\bar{\boldsymbol{c}}$ recovers $\boldsymbol{c}_\alpha$. Finally, we apply $\mathrm{dec}_{\mathrm{Gray}}$ to $\boldsymbol{c}_\alpha[0, k_R-1]$ to recover α and hence, the output a is indeed α.

Case 2: $\bar{i} \in [\ell_p, \ell_p+mk_R-1]$. By shifting $\boldsymbol{w}^\circ$ to the left $\hat{i}$ positions, we obtain the sequence $\boldsymbol{p}\boldsymbol{s}_{\alpha+1}[0, \bar{i}-\ell_p-1]\boldsymbol{s}_\alpha[\bar{i}-\ell_p, n'-1]$. So, $d_H(\boldsymbol{v}, \boldsymbol{p}\boldsymbol{s}_{\alpha+1}[0, \bar{i}-\ell_p-1]\boldsymbol{s}_\alpha[\bar{i}-\ell_p, n'-1]) = e$.

Set $j = \lfloor (\bar{i}-\ell_p)/m \rfloor$, and so, $j < k_R$. Note that there may be a subword of length m which covers both the tail of $\boldsymbol{s}_{\alpha+1}[0, \bar{i}-\ell_p-1]$ and the head of $\boldsymbol{s}_\alpha[\bar{i}-\ell_p, n'-1]$. Hence, we have $d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}_{\alpha+1}[0, j]\boldsymbol{c}_\alpha[j+1, n_R-1]) \leq e+1$. It follows that

$$\begin{aligned} d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}_\alpha) &= d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}_\alpha[0, j]\boldsymbol{c}_\alpha[j+1, n_R-1]) \\ &\leq d_H(\bar{\boldsymbol{c}}, \boldsymbol{c}_{\alpha+1}[0, j]\boldsymbol{c}_\alpha[j+1, n_R-1]) + 1 \\ &\leq e+2 \leq (d_R-1)/2. \end{aligned}$$

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Similar to Case 1, we apply $\mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)}$ to $\bar{\boldsymbol{c}}$ to recover $\boldsymbol{c}_\alpha$ and then apply $\mathrm{dec}_{\mathrm{Gray}}$ to recover $a=\alpha$.

Case 3: $\bar{i}\in[\ell_p+mk_R,\ell_p+mk_R+(d+1)(m+d)-1]$. Similar to Case 2, we have

$$d_H(\boldsymbol{v},\boldsymbol{p}\boldsymbol{s}_{\alpha+1}[0,\bar{i}-\ell_p-1]\boldsymbol{s}_\alpha[\bar{i}-\ell_p,n'-1])\le e.$$

Due to the range of $\bar{i}$, $\boldsymbol{s}_{\alpha+1}[0,\bar{i}-\ell_p-1]$ contains the subword $\boldsymbol{s}_{\alpha+1}[0,mk_R-1]$ and $\boldsymbol{s}_\alpha[\bar{i}-\ell_p,n'-1]$ contains the subword $\boldsymbol{s}_\alpha[mk_R+(d+1)(m+d),n'-1]$. It follows that the distance between the shortened vector

$$\bar{\boldsymbol{c}}^s=\bar{\boldsymbol{c}}[0,k_R-1]\bar{\boldsymbol{c}}[k_R+(d+1),n_R-1].$$

and the vector

$$\boldsymbol{c}_{\alpha+1}[0,k_R-1]\boldsymbol{c}_\alpha[k_R+(d+1),n_R-1]$$

is no more than e. Since $d_H(\boldsymbol{c}_\alpha[0,k_R-1],\boldsymbol{c}_{\alpha+1}[0,k_R-1])\le 1$, we have

$$d_H(\bar{\boldsymbol{c}}^s,\boldsymbol{c}_\alpha[0,k_R-1]\boldsymbol{c}_\alpha[k_R+(d+1),n_R-1])\le e+1.$$

The shortened vector $\boldsymbol{c}_\alpha[0,k_R-1]\boldsymbol{c}_\alpha[k_R+(d+1),n_R-1]$ can be treated as a codeword of a Reed-Solomon code of length $n_R-(d+1)$ and dimension k_R. Since $e+1\le(n_R-(d+1)-k_R)/2$, we apply the decoding algorithm $\mathrm{dec}_{\mathrm{RS}}^{(n_R-(d+1),k_R)}$ to recover $\boldsymbol{c}_\alpha[0,k_R-1]$. As before, we apply $\mathrm{dec}_{\mathrm{Gray}}$ to recover α.

Case 4: $\bar{i}\in[\ell_p+mk_R+(d+1)(m+d),n-1]$. In this case, we consider the shortened vector $\bar{\boldsymbol{c}}^s=\bar{\boldsymbol{c}}[0,k_R+(d+1)-1]$. Similar to Case 3, we have

$$d_H(\bar{\boldsymbol{c}}^s,\boldsymbol{c}_{\alpha+1}[0,k_R+(d+1)-1])\le e.$$

As before, we can $\mathrm{dec}_{\mathrm{RS}}^{(n_R-(d+1),k_R)}$ to recover $\boldsymbol{c}_{\alpha+1}[0,k_R-1]$ and hence, recover $\alpha+1$.

We analyse the running time. To determine $\hat{i}$, we require $\ell_p n$ comparisons. Next, the Reed-Solomon decoding $\mathrm{dec}_{\mathrm{RS}}^{(n_R,k_R)}$ runs in $O(n_R^3)=O(n^3)$ time. Finally, the decoding of Gray codes $\mathrm{dec}_{\mathrm{Gray}}$ runs in $O(k_R\log^2 q)=O(nm^2)=O(n\log^2 n)$ time. Therefore, Algorithm 1 computes the location in $O(n^3)$ time. □

4 Binary Robust Positioning Arrays with Constant Distance

We generalize Construction 1A to produce binary RPAs of dimensions $n_1\times n_2$ with constant distance. To this end, we require the following number theoretic result.

LEMMA 4.1. (BAKER ET AL. [2]) *Let $\theta=0.525$. There exists x_0 such that for every $x\ge x_0$, the interval $[x-x^\theta,x]$ contains a prime.*

Fix d and suppose that $n_1=O(n_2^t)$ with t being a finite number. Set $m=\frac{\log(n_1n_2)}{1-\theta}$ and $r=\sum_{i=d}^m\binom{m}{i}$, where $\theta=0.525$. Lemma 4.1 then provides a prime q such that $r-r^\theta\le q\le r$. Take an arbitrary injective map ψ from $\mathbb{F}_q$ to $\mathbb{F}_2^m$ such that $w_H(\psi(x))\ge d$ for all $x\in\mathbb{F}_q$. In other words, ψ maps symbols in $\mathbb{F}_q$ to binary sequences of length m with weight at least d. For a vector $\boldsymbol{x}=x_1x_2\cdots x_n\in\mathbb{F}_q^n$, let $\psi(\boldsymbol{x})=\psi(x_1)\psi(x_2)\cdots\psi(x_n)$.

Suppose that n_2 is divisible by m. Set $n_R\triangleq n_1(n_2/m)-4$ and $k_R\triangleq n_R-2(d+7)$. Then we have $k_R<n_R<q$ and set $M\triangleq q^{k_R/2}$. Now we provide our construction of robust positioning arrays.

Construction 2. Let $\mathcal{G}=(\boldsymbol{\sigma}_0,\boldsymbol{\sigma}_1,\ldots,\boldsymbol{\sigma}_{M-1})$ be a $(k_R/2,q)$-Gray code and consider a Reed-Solomon code of length n_R and dimension k_R over $\mathbb{F}_q$. For each $0\le i,j\le M-1$, set $\boldsymbol{c}_{ij}=\mathrm{enc}_{\mathrm{RS}}^{(n_R,k_R)}(\boldsymbol{\sigma}_i\boldsymbol{\sigma}_j)$.

Let $\boldsymbol{p}=0^k\boldsymbol{u}$ where $k=4m-\ell$ and $\boldsymbol{u}$ is the d-auto-cyclic vector provided in (3.1). For each $\boldsymbol{c}_{ij}$, the concatenation $\boldsymbol{p}\psi(\boldsymbol{c}_{ij})$ has length n_1n_2 as $n_R=n_1(n_2/m)-4$. Then let $\mathbb{A}_{ij}$ be the $n_1\times n_2$ array whose rows can be concatenated to form $\boldsymbol{p}\psi(\boldsymbol{c}_{ij})$ (see Fig. 3).

Finally, construct a large array $\mathbb{A}$ as

$$\mathbb{A}=\begin{pmatrix}\mathbb{A}_{00}&\mathbb{A}_{01}&\cdots&\mathbb{A}_{0,M-1}\\ \mathbb{A}_{10}&\mathbb{A}_{11}&\cdots&\mathbb{A}_{1,M-1}\\ \vdots&\vdots&\ddots&\vdots\\ \mathbb{A}_{M-1,0}&\mathbb{A}_{M-1,1}&\cdots&\mathbb{A}_{M-1,M-1}\end{pmatrix}.$$

For an array $\mathbb{M}=(m_{i,j})$, we use $\mathbb{M}[i_0,i_0+a-1][j_0,j_0+b-1]$ to denote the $a\times b$ cyclical subarray of $\mathbb{M}$ whose top-left cell is $m_{i_0j_0}$. The following result is an analogue to Lemma 3.1 and helps to locate the modular position efficiently.

LEMMA 4.2. *Consider the subarray $\mathbb{W}=\mathbb{A}[i_0,i_0+n_1-1][j_0,j_0+n_2-1]$ in $\mathbb{A}$. Pick $i\in[\![n_1]\!]$ and $j\in[\![n_2]\!]$. Then the following hold.*

(i) If $i+i_0\equiv 0\pmod{n_1}$ and $j+j_0\pmod{n_2}$, then $\mathbb{W}[i,i][j,j+4m-1]=\boldsymbol{p}$.

(ii) If $i+i_0\not\equiv 0\pmod{n_1}$ or $j+j_0\not\equiv 0\pmod{n_2}$, then $d_H(\mathbb{W}[i,i][j,j+4m-1],\boldsymbol{p})\ge d$.

Proof. Let $\hat{i}\in[n_1]$ and $\hat{j}\in[n_2]$ such that $\hat{i}+i_0\equiv 0\pmod{n_1}$ and $\hat{j}+j_0\equiv 0\pmod{n_2}$. We consider the array $\mathbb{V}$, which is obtained by shifting $\mathbb{W}$ to the up cyclically $\hat{i}$ times and to the left cyclically $\hat{j}$ times. Then $\mathbb{V}[0,0][0,4m-1]=\boldsymbol{p}$, see Fig. 4, and it suffices to show $d_H(\mathbb{V}[i,i][j,j+4m-1],\boldsymbol{p})\ge d$ when $i\in[1,n_1-1]$ or $j\in[1,n_2-1]$.

For $i\in[1,n_1-1]$, since $k=4m-\ell>3m$, $\mathbb{V}[i,i][j,j+k-1]$ must contain a length-m vector $\psi(x_0)$

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

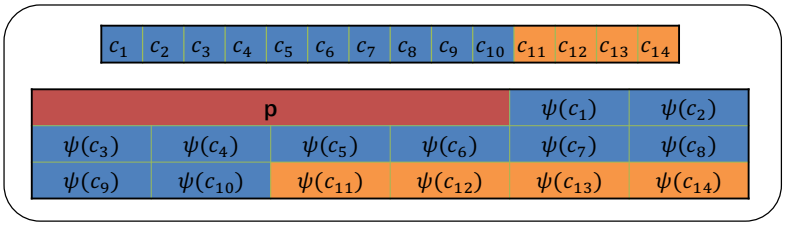


Figure 3: A codeword from a Reed-Solomon code of length 14 and dimension 10 and its corresponding $n_1 \times n_2$ array with $n_1 = 3$ and $n_2 = 6m$. The blue cells represent the message bits and the yellow cells represent the check bits.

for some $x_0 \in \mathbb{F}_q$ (see Fig. 4). Observe that $\psi(x_0)$ has weight $\geq d$. Hence, we have

$$\begin{aligned} &d_H(\mathbb{V}[i,i][j,j+4m-1], \boldsymbol{p}) \\ &\quad \geq d_H(\mathbb{V}[i,i][j,j+k-1], \boldsymbol{p}[0,k-1]) \\ &\quad = d_H(\mathbb{V}[i,i][j,j+k-1], 0^k) \geq d. \end{aligned}$$

For $i = 0$ and $j \in [1, n_2 - 1]$, the proof follows from Lemma 3.1. □

We regard an array of size $a \times (bm)$ as a $a \times b$ partitioned matrix with each block being a vector of length m. Given a pair of $a \times (bm)$ arrays $\mathbb{M}_1$ and $\mathbb{M}_2$, we denote the Hamming distance of their corresponding partitioned matrices as $d_B(\mathbb{M}_1, \mathbb{M}_2)$. In other words, $d_B(\mathbb{M}_1, \mathbb{M}_2)$ counts the number of different blocks in $\mathbb{M}_1$ and $\mathbb{M}_2$. Therefore, in Construction 2, we have

$$d_H(\mathbb{A}_{ij}, \mathbb{A}_{i'j'}) \geq d_B(\mathbb{A}_{ij}, \mathbb{A}_{i'j'}) = d_H(\boldsymbol{c}_{ij}, \boldsymbol{c}_{i'j'}).$$

For a pair of $a \times b$ arrays $\mathbb{M}_1$ and $\mathbb{M}_2$ with b not divisible by m, we can repeat the last columns to form two arrays $\mathbb{M}_1'$ and $\mathbb{M}_2'$ of size $a \times (\lceil b/m \rceil m)$. Then denote $d_B(\mathbb{M}_1, \mathbb{M}_2) := d_B(\mathbb{M}_1', \mathbb{M}_2')$. Hence, $d_B(\mathbb{M}_1, \mathbb{M}_2)$ counts the number of different (truncated) blocks in $\mathbb{M}_1$ and $\mathbb{M}_2$.

LEMMA 4.3. *For any two subarrays* $\mathbb{W} = \mathbb{A}[i, i+n_1-1][j, j+n_2-1]$ *and* $\mathbb{W}' = \mathbb{A}[i', i'+n_1-1][j', j'+n_2-1]$ *with* $i \equiv i' \pmod{n_1}$ *and* $j \equiv j' \pmod{n_2}$, *the Hamming distance between them is at least* d.

Proof. Suppose that $i = an + \bar{i}$ and $i' = a'n + \bar{i}$ for some $\bar{i} \in [\![n_1]\!]$, and $j = bn + \bar{j}$ and $j' = b'n + \bar{j}$ for some $\bar{j} \in [\![n_2]\!]$. Let $\hat{i} \in [\![n_1]\!]$ and $\hat{j} \in [\![n_2]\!]$ be the integers such that $\bar{i} + \hat{i} \equiv 0 \pmod{n_1}$ and $\bar{j} + \hat{j} \equiv 0 \pmod{n_2}$. Shift $\mathbb{W}$ cyclically to the up $\hat{i}$ times and to the left $\hat{j}$ times and denote the resulting array as $\mathbb{V}$. Similarly, let $\mathbb{V}'$ be the corresponding shifted array of $\mathbb{W}'$. Then $d_H(\mathbb{W}, \mathbb{W}') = d_H(\mathbb{V}, \mathbb{V}')$. Since $n_1 = O(n_2^t)$ with t being a finite number, we have that $(n_R - k_R)m < n_2$. It follows that the check bits of $\mathbb{A}_{\alpha\beta}$ appear in the last row $\mathbb{A}_{\alpha\beta}[n_1-1, n_1-1][n_2-(n_R-k_R)m, n_2-1]$. To estimate $d_H(\mathbb{V}, \mathbb{V}')$, we proceed in three cases, depending on where the check bits of $\mathbb{V}$ and $\mathbb{V}'$ come from.

Case 1: $\bar{j} \in [0, n_2 - (n_R - k_R)m - 1]$. As shown in Fig. 5(a), we partition $\mathbb{W}$ into four blocks by setting

$$\begin{aligned} \mathbb{W}_{\mathrm{I}} &= \mathbb{W}[0, \hat{i}-1][0, \hat{j}-1], \\ \mathbb{W}_{\mathrm{II}} &= \mathbb{W}[0, \hat{i}-1][\hat{j}, n_2-1], \\ \mathbb{W}_{\mathrm{III}} &= \mathbb{W}[\hat{i}, n_1-1][0, \hat{j}-1], \\ \mathbb{W}_{\mathrm{IV}} &= \mathbb{W}[\hat{i}, n_1-1][\hat{j}, n_2-1]; \end{aligned}$$

and partition $\mathbb{A}_{ab}$ into four blocks by setting

$$\begin{aligned} \mathbb{A}_{\mathrm{I}} &= \mathbb{A}_{ab}[\bar{i}, n_1-1][\bar{j}, n_2-1], \\ \mathbb{A}_{\mathrm{II}} &= \mathbb{A}_{ab}[\bar{i}, n_1-1][0, \bar{j}-1], \\ \mathbb{A}_{\mathrm{III}} &= \mathbb{A}_{ab}[0, \bar{i}-1][\bar{j}, n_2-1], \\ \mathbb{A}_{\mathrm{IV}} &= \mathbb{A}_{ab}[0, \bar{i}-1][0, \bar{j}-1]. \end{aligned}$$

Then we have

$$\mathbb{V} = \begin{pmatrix} \mathbb{W}_{\mathrm{IV}} & \mathbb{W}_{\mathrm{III}} \\ \mathbb{W}_{\mathrm{II}} & \mathbb{W}_{\mathrm{I}} \end{pmatrix}, \mathbb{A}_{ab} = \begin{pmatrix} \mathbb{A}_{\mathrm{IV}} & \mathbb{A}_{\mathrm{III}} \\ \mathbb{A}_{\mathrm{II}} & \mathbb{A}_{\mathrm{I}} \end{pmatrix},$$

and $\mathbb{W}_{\mathrm{I}} = \mathbb{A}_{\mathrm{I}}$.

Notice that all these blocks, except $\mathbb{W}_{\mathrm{I}}$ and $\mathbb{A}_{\mathrm{I}}$, do not contain check bits, and $(\boldsymbol{\sigma}_0, \boldsymbol{\sigma}_1 \ldots, \boldsymbol{\sigma}_{M-1})$ is a Gray code. It follows that

$$\begin{aligned} d_B(\mathbb{W}_{\mathrm{II}}, \mathbb{A}_{\mathrm{II}}) &\leq d_H(\boldsymbol{\sigma}_a \boldsymbol{\sigma}_b, \boldsymbol{\sigma}_a \boldsymbol{\sigma}_{b+1}) \leq 1, \\ d_B(\mathbb{W}_{\mathrm{III}}, \mathbb{A}_{\mathrm{III}}) &\leq d_H(\boldsymbol{\sigma}_a \boldsymbol{\sigma}_b, \boldsymbol{\sigma}_{a+1} \boldsymbol{\sigma}_b) \leq 1, \\ \text{and } \ d_B(\mathbb{W}_{\mathrm{IV}}, \mathbb{A}_{\mathrm{IV}}) &\leq d_H(\boldsymbol{\sigma}_a \boldsymbol{\sigma}_b, \boldsymbol{\sigma}_{a+1} \boldsymbol{\sigma}_{b+1}) \leq 2. \end{aligned}$$

So,

$$\begin{aligned} d_B(\mathbb{V}, \mathbb{A}_{ab}) &\leq d_B(\mathbb{W}_{\mathrm{I}}, \mathbb{A}_{\mathrm{I}}) + d_B(\mathbb{W}_{\mathrm{II}}, \mathbb{A}_{\mathrm{II}}) \\ &\quad + d_B(\mathbb{W}_{\mathrm{III}}, \mathbb{A}_{\mathrm{III}}) + d_B(\mathbb{W}_{\mathrm{IV}}, \mathbb{A}_{\mathrm{IV}}) \leq 4. \end{aligned}$$

With the same argument, we can get $d_B(\mathbb{V}', \mathbb{A}_{a'b'}) \leq 4$. Hence,

$$\begin{aligned} d_H(\mathbb{V}, \mathbb{V}') &\geq d_B(\mathbb{V}, \mathbb{V}') \geq d_B(\mathbb{A}_{ab}, \mathbb{A}_{a'b'}) - 8 \\ &= d_H(\boldsymbol{c}_{ab}, \boldsymbol{c}_{a'b'}) - 8 \geq d. \end{aligned}$$

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

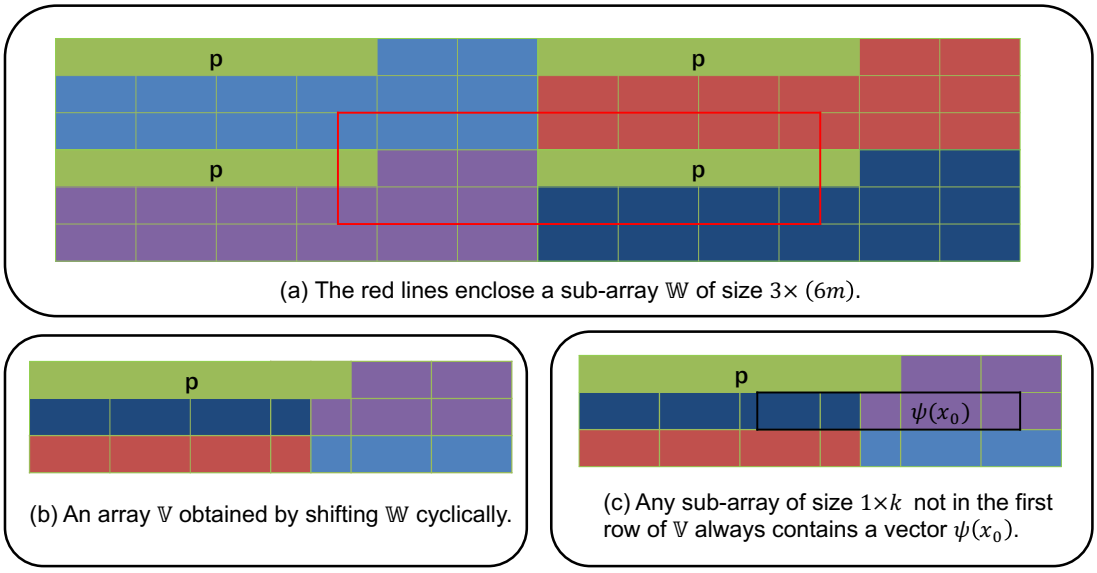


Figure 4: An example with $n_1 = 3$ and $n_2 = 6m$ to illustrate the proof of Lemma 4.2. **(a)** The red lines enclose the subarray $\mathbb{W} = \mathbb{A}[2,5][3.5m, 9.5m-1]$. **(a)** Shifting $\mathbb{W}$ to the up one times and to the left $2.5m$ times, we got the array $\mathbb{V}$. **(c)** Any subarray $\mathbb{V}[i,i][j,j+k-1]$ with $i \neq 0$ always contains a vector $\psi(x_0)$ for some $x_0 \in \mathbb{F}_q$.

Case 2: $\bar{j} \in [n_2-(n_R-k_R)m, n_2-(n_R-k_R)m/2-1]$. For $\alpha,\beta \in [\![M]\!]$, we change the bits in the block $\mathbb{A}_{\alpha\beta}[n_1-1,n_1-1][n_2-(n_R-k_R)m, n_2-(n_R-k_R)m/2-1]$ to one and denote the resulting array as $\bar{\mathbb{A}}_{\alpha\beta}$. Let $\bar{\boldsymbol{c}}_{\alpha\beta}$ be the shortened codeword of $\boldsymbol{c}_{\alpha,\beta}$ by deleting the subword $\boldsymbol{c}_{\alpha\beta}[k_R, n_R-(n_R-k_R)/2-1]$ (see Fig. 5(b)). Then we have

$$\begin{aligned} d_B(\bar{\mathbb{A}}_{\alpha\beta}, \bar{\mathbb{A}}_{\alpha'\beta'}) &= d_H(\bar{\boldsymbol{c}}_{\alpha\beta}, \bar{\boldsymbol{c}}_{\alpha'\beta'}) \\ &\geq (n_R-(n_R-k_R)/2) - k_R + 1 \geq d+8. \end{aligned} \tag{4.2}$$

Now, let $\bar{\mathbb{W}}$, $\bar{\mathbb{W}}'$, $\bar{\mathbb{V}}$ and $\bar{\mathbb{V}}'$ be the corresponding arrays of $\mathbb{W}$, $\mathbb{W}'$, $\mathbb{V}$ and $\mathbb{V}'$ with some check bits being changed to one. As in Case 1, we can show that

$$d_B(\bar{\mathbb{V}}, \bar{\mathbb{A}}_{ab}) \leq 4 \text{ and } d_B(\bar{\mathbb{V}}', \bar{\mathbb{A}}'_{ab}) \leq 4. \tag{4.3}$$

The only difference is that $\bar{\mathbb{W}}_{\mathrm{II}}$, $\bar{\mathbb{A}}_{\mathrm{II}}$, $\bar{\mathbb{W}}'_{\mathrm{II}}$, and $\bar{\mathbb{A}}'_{\mathrm{II}}$ may contain the check bits. However, these bits are set to one, so we have $d_B(\bar{\mathbb{W}}_{\mathrm{II}}, \bar{\mathbb{A}}_{\mathrm{II}}) \leq d_H(\boldsymbol{\sigma}_a\boldsymbol{\sigma}_b, \boldsymbol{\sigma}_a\boldsymbol{\sigma}_{b+1}) \leq 1$, see Fig. 5(c).

It follows from (4.2) and (4.3) that

$$d_B(\bar{\mathbb{V}}, \bar{\mathbb{V}}') \geq d_B(\bar{\mathbb{A}}_{ab}, \bar{\mathbb{A}}_{a'b'}) - 8 \geq d,$$

and then

$$d_H(\mathbb{V}, \mathbb{V}') \geq d_H(\bar{\mathbb{V}}, \bar{\mathbb{V}}') \geq d_B(\bar{\mathbb{V}}, \bar{\mathbb{V}}') \geq d.$$

Case 3: $\bar{j} \in [n_2-(n_R-k_R)m/2, n_2-1]$. For $\alpha,\beta \in [\![M]\!]$, we change the bits in the block $\mathbb{A}_{\alpha\beta}[n_1-1,n_1-1][n_2-(n_R-k_R)m/2, n_2-1]$ to one and denote the resulting array as $\tilde{\mathbb{A}}_{\alpha\beta}$. Let $\tilde{\boldsymbol{c}}_{\alpha\beta}$ be the shortened codeword of $\boldsymbol{c}_{\alpha,\beta}$ by deleting the last $(n_R-k_R)/2$ bits.

Now, let $\tilde{\mathbb{W}}$, $\tilde{\mathbb{W}}'$, $\tilde{\mathbb{V}}$ and $\tilde{\mathbb{V}}'$ be the corresponding arrays of $\mathbb{W}$, $\mathbb{W}'$, $\mathbb{V}$ and $\mathbb{V}'$. Again we use the strategy in Case 1 to show that

$$\begin{aligned} d_H(\mathbb{V}, \mathbb{V}') &\geq d_H(\tilde{\mathbb{V}}, \tilde{\mathbb{V}}') \geq d_B(\tilde{\mathbb{V}}, \tilde{\mathbb{V}}') \\ &\geq d_B(\tilde{\mathbb{A}}_{a,b+1}, \tilde{\mathbb{A}}_{a',b'+1}) - 8 \geq d. \end{aligned}$$

We note that in this case we need to partition $\tilde{\mathbb{A}}_{a,b+1}$ instead of $\tilde{\mathbb{A}}_{ab}$, see Fig. 5(d). □

THEOREM 4.1. *The array $\mathbb{A}$ in Construction 2 is an $(n_1 \times n_2, d)$-RPA.*

Proof. Let $\mathbb{W}$ and $\mathbb{W}'$ be two distinct subarrays of size $n_1 \times n_2$ in $\mathbb{A}$. Assume that $\mathbb{W} = \mathbb{A}[i, i+n_1-1][j, j+n_2-1]$ and $\mathbb{W}' = \mathbb{A}[i', i'+n_1-1][j', j'+n_2-1]$, where $(i,j) \neq (i',j')$. From Lemma 4.3, we have $d_H(\mathbb{W}, \mathbb{W}') \geq d$ when $i \equiv i' \pmod{n_1}$ and $j \equiv j' \pmod{n_2}$. Now we consider the case where $i \not\equiv i' \pmod{n_1}$ or $j \not\equiv j' \pmod{n_2}$. Let $\hat{i} \in [n_1]$ and $\hat{j} \in [n_2]$ such that $i+\hat{i} \equiv 0 \pmod{n_1}$ and $j+\hat{j} \equiv 0 \pmod{n_2}$. So we have $i'+\hat{i} \not\equiv 0 \pmod{n_1}$ or $j'+\hat{j} \not\equiv 0 \pmod{n_2}$. It follows from Lemma 4.2 that $\mathbb{W}[\hat{i},\hat{i}][\hat{j}, \hat{j}+4m-1] = \boldsymbol{p}$ and $d_H(\mathbb{W}'[\hat{i},\hat{i}][\hat{j}, \hat{j}+4m-1], \boldsymbol{p}) \geq d$. Hence

$$\begin{aligned} &d_H(\mathbb{W}, \mathbb{W}') \\ &\geq d_H(\mathbb{W}[\hat{i},\hat{i}][\hat{j}, \hat{j}+4m-1], \mathbb{W}'[\hat{i},\hat{i}][\hat{j}, \hat{j}+4m-1]) \\ &\geq d, \end{aligned}$$

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

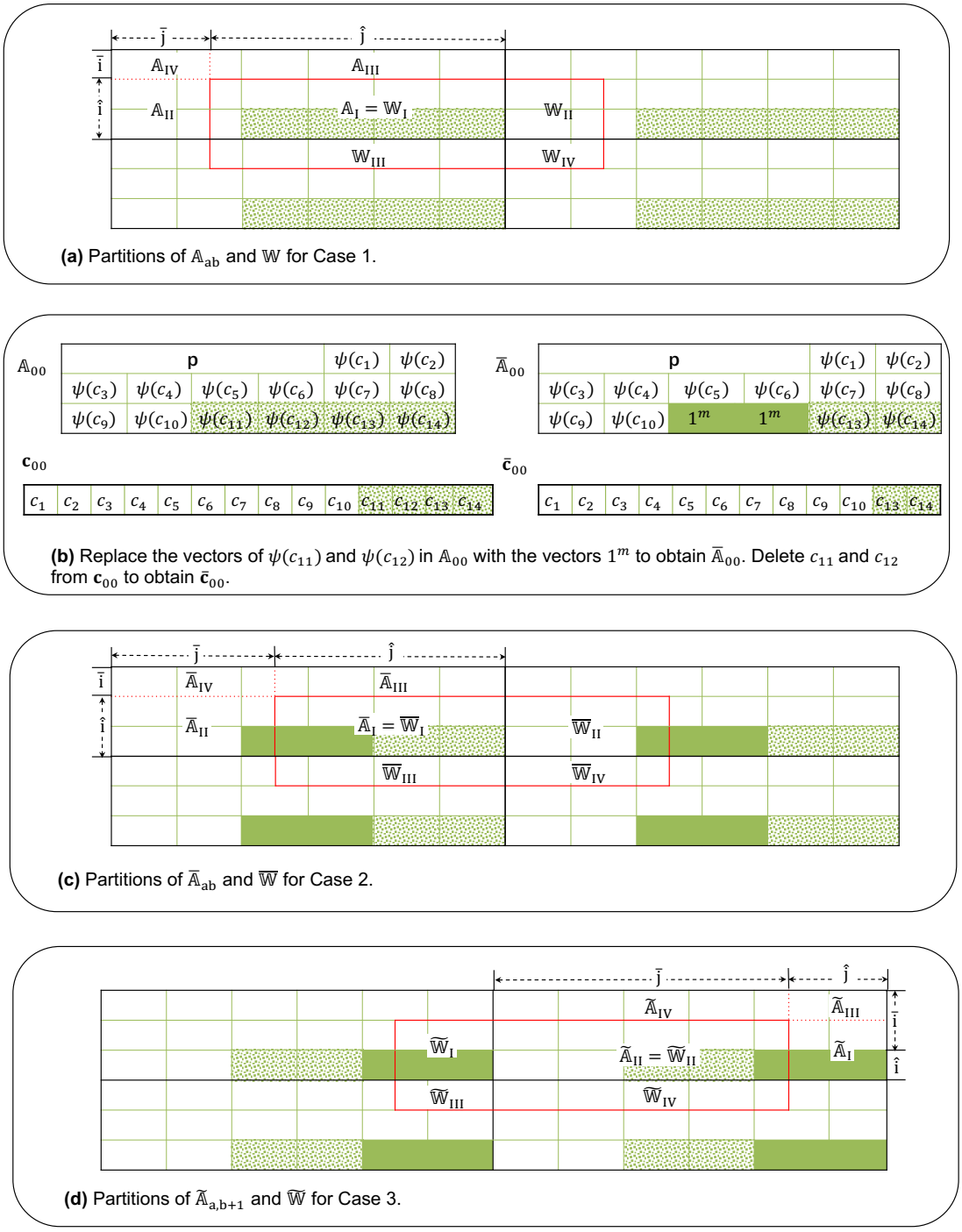


Figure 5: An example with $n_1 = 3$ and $n_2 = 6m$ to illustrate the proof of Lemma 4.3. The black lines enclose the arrays $\mathbb{A}_{ab}$, $\mathbb{A}_{a,b+1}$, $\mathbb{A}_{a+1,b}$ and $\mathbb{A}_{a+1,b+1}$. The red lines enclose the subarray $\mathbb{W}$. The empty blocks represent the vectors of message bits. The blocks with dots represent the vectors of check bits. The solid blocks represent the vectors 1^m.

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

which completes the proof. □

COROLLARY 4.1. *For sufficient large n_1, there is an $(n_1 \times n_2, d)$-RPA with redundancy at most*

$$4.21d \log n_1 n_2 + 35.79 \log(n_1 n_2) + o(1).$$

Proof. The RPA $\mathbb{A}$ in Construction 2 has dimensions $(n_1 M) \times (n_2 M)$, where $M = q^{k_R/2}$. So, its redundancy is given by

$$n_1 n_2 - \log(n_1 n_2 M^2) = n_1 n_2 - k_R \log q - \log(n_1 n_2),$$

where $k_R = n_R - 2(d+7) = n_1 n_2/m - 2d - 18$, and

$$\begin{aligned} \log q &\geq \log(r - r^\theta) \\ &= \log r + \log\left(1 - \frac{1}{r^{1-\theta}}\right) \\ &\geq \log r - \frac{\log e}{r^{1-\theta} - 1}. \end{aligned} \tag{4.4}$$

Recall that $\theta = 0.525$, and so, $m = \log(n_1 n_2)/(1-\theta) \geq (3/2)\log(n_1 n_2)$. It follows that

$$\begin{aligned} r = \sum_{i=d}^{m} \binom{m}{i} &= 2^m \left[1 - \sum_{i=0}^{d-1} \binom{m}{i} \frac{1}{2^m}\right] \\ &\geq 2^m \left[1 - \exp\left(-\frac{2(m/2 - d + 1)^2}{m}\right)\right] \\ &\geq 2^m \left(1 - e^{-\frac{2m}{3 \log e}}\right) \geq 2^m \left(1 - \frac{1}{n_1 n_2}\right). \end{aligned}$$

Then we have

$$\log r \geq m + \log\left(1 - \frac{1}{n_1 n_2}\right) \geq m - \frac{\log e}{n_1 n_2 - 1}. \tag{4.5}$$

On the other hand,

$$r^{1-\theta} \geq \left(\frac{2^m}{2}\right)^{1-\theta} = \frac{n_1 n_2}{2^{1-\theta}}. \tag{4.6}$$

Combining (4.4), (4.5) and (4.6), we get

$$\log q \geq m - O\left(\frac{1}{n_1 n_2}\right).$$

Hence, the redundancy of $\mathbb{A}$ is at most

$$\begin{aligned} n_1 n_2 - &\left(\frac{n_1 n_2}{m} - 2d - 18\right)\left(m - O\left(\frac{1}{n_1 n_2}\right)\right) \\ &- \log(n_1 n_2) \\ &= \frac{2d}{1-\theta} \log(n_1 n_2) + \frac{18}{1-\theta} \log(n_1 n_2) \\ &- \log(n_1 n_2) + o(1) \\ &\approx 4.21d \log n_1 n_2 + 36.89 \log(n_1 n_2) + o(1). \end{aligned}$$ □

To conclude, we provide an efficient locating algorithm for the array $\mathbb{A}$ in Construction 2. Let χ be a map from $\mathbb{F}_2^m$ to $\mathbb{F}_q$ such that $\chi(\psi(x)) = x$ for all $x \in \mathbb{F}_q$ and $\chi(\boldsymbol{v}) = 0$ for all $\boldsymbol{v} \notin \{\psi(x) : x \in \mathbb{F}_q\}$.

We describe the algorithm in Algorithm 2. Using Lemma 4.2 and following the proof of Lemma 4.3, it is easy to show that Algorithm 2 can determine the position of $\mathbb{W}$ in the array $\mathbb{A}$ as long as the number of errors is at most $(d-1)/2$ errors. Furthermore, following the proof of Theorem 3.4, we see that Algorithm 2 determines the position in $O((n_1 n_2)^3)$ time.

Algorithm 2 Locating algorithm for the array $\mathbb{A}$ in Construction 2

Input: a subarray $\mathbb{W}$ of dimensions $n_1 \times n_2$
Output: a position $(i, j) \triangleq (an_1 + \bar{i}, bn_2 + \bar{j})$ such that $d_H(\boldsymbol{S}[i, i+n_1-1][j, j+n_2-1], \boldsymbol{w}) \leq (d-1)/2$

$m \leftarrow \frac{\log(n_1 n_2)}{1-\theta}$
$n_R \leftarrow n_1(n_2/m) - 4$
$k_R \leftarrow n_R - 2(d+7)$

$(\hat{i}, \hat{j}) \leftarrow$ unique tuple such that $d_H(\boldsymbol{w}[\hat{i}, \hat{i}][\hat{j}, \hat{j} + 4m - 1], \boldsymbol{p}) \leq (d-1)/2$
Set $\bar{i} \in [\![n_1]\!]$ such that $\bar{i} + \hat{i} \equiv 0 \pmod{n_1}$
Set $\bar{j} \in [\![n_2]\!]$ such that $\bar{j} + \hat{j} \equiv 0 \pmod{n_2}$

$\mathbb{V} \leftarrow$ the array obtained by shifting $\mathbb{W}$ cyclically to the up $\hat{i}$ times and to the left $\hat{j}$ times
$\boldsymbol{v} \leftarrow$ the concatenation of the rows of $\mathbb{V}$
$\boldsymbol{u} \leftarrow \chi(\boldsymbol{v}[4m, 5m-1])\chi(\boldsymbol{v}[5m, 6m-1])\chi(\boldsymbol{v}[6m, 7m-1]) \cdots \chi(\boldsymbol{v}[n_1 n_2 - m, n_1 n_2 - 1])$

if $\bar{j} \in [0, n_2 - 2(d+7)m - 1]$ **then**
  $\boldsymbol{c} \leftarrow \mathrm{dec}_{\mathrm{RS}}^{(n_R, k_R)}(\boldsymbol{u})$
else if $\bar{j} \in [n_2 - 2(d+7)m, n_2 - (d+7)m - 1]$ **then**
  $\boldsymbol{u}^s \leftarrow$ the shortened codeword $\boldsymbol{u}[0, k_R - 1]\boldsymbol{u}[k_R + (d+7), n_R - 1]$
  $\boldsymbol{c} \leftarrow \mathrm{dec}_{\mathrm{RS}}^{(n_R - (d+7), k_R)}(\boldsymbol{u}^s)$
else
  $\boldsymbol{u}^s \leftarrow$ the shortened codeword $\bar{\boldsymbol{c}}[0, k_R + d + 6]$
  $\boldsymbol{c} \leftarrow \mathrm{dec}_{\mathrm{RS}}^{(n_R - (d+7), k_R)}(\boldsymbol{u}^s)$
$a \leftarrow \mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{c}[0, k_R/2 - 1])$
if $\bar{j} \in [0, n_2 - (d+7)m - 1]$ **then**
  $b \leftarrow \mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{c}[k_R/2, k_R - 1])$
else
  $b + 1 \leftarrow \mathrm{dec}_{\mathrm{Gray}}(\boldsymbol{c}[k_R/2, k_R - 1])$
return $(an_1 + \bar{i}, bn_2 + \bar{j})$

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited

5 Conclusion

We construct binary positioning patterns, equipped with efficient locating algorithms, that are robust to a constant number of errors. Our strategy is based on d-auto-cyclic vectors, Reed-Solomon codes and Gray codes, and we reduce the number of redundancies as compared to previous constructions. In the locating algorithms, the d-auto-cyclic vectors are used as markers to locate the relative position. This information, together with the property of Gray codes, allows one to leverage the well-known fast decoding of Reed-Solomon codes to quickly identify the location.

References

[1] *Microsoft mulls a stylus for any screen*, http://www.technologyreview.com/news/428521/microsoftmulls-a-stylus-for-any-screen/.

[2] R. C. Baker, G. Harman, and J. Pintz, *The difference between consecutive primes, II*, Proc. London Math. Society, 83 (2001), pp. 532–562.

[3] R. Berkowitz and S. Kopparty, *Robust positioning patterns*, in Proc. ACM-SIAM Symp. Discrete Algorithms, Society for Industrial and Applied Mathematics, 2016, pp. 1937–1951.

[4] A. M. Bruckstein, T. Etzion, R. Giryes, N. Gordon, R. J. Holt, and D. Shuldiner, *Simple and robust binary self-location patterns*, IEEE Trans. Inform. Theory, 58 (2012), pp. 4884–4889.

[5] J. Dai and C. K. R. Chung, *Touchscreen everywhere: on transferring a normal planar surface to a touch-sensitive display*, IEEE Trans. Cybernetics, 44 (2014), pp. 1383–1396.

[6] T. Etzion, *Constructions for perfect maps and pseudo-random arrays*, IEEE Trans. Inform. Theory, 34 (1988), pp. 1308–1316.

[7] J. Geng, *Structured-light 3D surface imaging: a tutorial*, Advances in Optics and Photonics, 3 (2011), pp. 128–160.

[8] D. J. Guan, *Generalized Gray codes with applications*, in Proc. Natl. Sci. Counc. Repub. China Part A Phys. Sci. Eng., vol. 22, 1998, pp. 841–848.

[9] M. Hagita, M. Matsumoto, F. Natsu, and Y. Ohtsuka, *Error correcting sequence and projective de Bruijn graph*, Graphs and Combinatorics, 24 (2008), pp. 185–194.

[10] P. V. Kumar and V. K. Wei, *Minimum distance of logarithmic and fractional partial m-sequences*, IEEE Trans. Inform. Theory, 38 (1992), pp. 1474–1482.

[11] M. Levy and E. Yaakobi, *Mutually uncorrelated codes for DNA storage*, IEEE Trans. Inform. Theory, 2018. Available at *DOI: 10.1109/TIT.2018.2873138*,

[12] F. J. MacWilliams and N. J. A. Sloane, *Pseudo-random sequences and arrays*, Proc. IEEE, 64 (1976), pp. 1715–1729.

[13] C. J. Mitchell, T. Etzion, and K. G. Paterson, *A method for constructing decodable de Bruijn sequences*, IEEE Trans. Inform. Theory, 42 (1996), pp. 1472–1478.

[14] K. G. Paterson, *Perfect maps*, IEEE Trans. Inform. Theory, 40 (1994), pp. 743–753.

[15] E. R. Scheinerman, *Determining planar location via complement-free de Brujin sequences using discrete optical sensors*, IEEE Trans. Robotics and Automation, 17 (2001), pp. 883–889.

[16] I. Szentandrasi, M. Zachariáš, J. Havel, A. Herout, M. Dubska, and R. Kajan, *Uniform marker fields: Camera localization by orientable de bruijn tori*, in Proc. IEEE Intl. Symp. Mixed and Augmented Reality, 2012, pp. 319–320.

[17] L. Wang, S. Hu, and O. Shayevitz, *Quickest sequence phase detection*, IEEE Trans. Inform. Theory, 63 (2017), pp. 5834–5849.

[18] L. R. Welch and E. R. Berlekamp, *Error correction for algebraic block codes*, Dec. 30 1986. US Patent 4,633,470.

[19] S. M. T. Yazdi, H. M. Kiah, R. Gabrys, and O. Milenkovic, *Mutually uncorrelated primers for DNA-based data storage*, IEEE Trans. Inform. Theory, 66 (2018), pp. 6283–6296.

Downloaded 01/26/19 to 182.19.246.40. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Copyright © 2019 by SIAM
Unauthorized reproduction of this article is prohibited